COMPTE RENDU DE TRAVAUX PRATIQUES DE CYBERSECURITE

Analyse réseau avec Wireshark

1. Introduction

Wireshark est un analyseur de trafic réseau open source qui permet de capturer et d'inspecter en temps réel les paquets circulant sur un réseau. Il décode les trames à tous les niveaux du modèle OSI et offre des filtres puissants pour isoler le trafic pertinent. C'est l'outil de référence en réseau et sécurité pour diagnostiquer des problèmes réseau ou détecter des comportements suspects.

L'analyse réseau constitue le fondement de toute démarche de sécurité informatique. Elle permet de comprendre ce qui circule réellement sur l'infrastructure, d'identifier des anomalies, de détecter des intrusions et de valider le comportement des équipements. **Sans visibilité sur le trafic réseau**, il est impossible de défendre efficacement un système d'information.

2. Description des manipulations

La manipulation a été fait en plusieurs étapes :

- Étapes 1 : lancer wireshark grâce commande wireshark
- Étapes 2 : Double cliqué sur l’interface réseau utiliser par le réseau qui est ici etho0
- Étapes 3 : Taper la commande nmap –Ss 192.168.56.40 pour avoir un trafic identifiable à analyser.
- Étapes 5 : appliquer un fire de son choix pour filtre le trafic réseaux. Exemples :tcp.flags.syn == 1 pour afficher uniquement les paquets SYN

3. Résultats obtenus

3.1 Capture du scan Nmap

Le scan Nmap a retourné les résultats suivants :

```
┌──(faustin㉿faustin)-[~]
└─$ nmap -sS 192.168.56.40
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-13 03:27 +0200
Nmap scan report for 192.168.56.40
Host is up (0.0028s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:74:9F:94 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.91 seconds
```

Interprétation : La cible .40 (serveur linux) affiche é ports ouverts.

3.2 Capture Wireshark — Trafic brut

No.	Time	Source	Destination	Protocol	Length	Info
1755	108.868330821	192.168.56.40	192.168.56.30	TCP	60	5802 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1756	108.868331157	192.168.56.40	192.168.56.30	TCP	60	19350 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1757	108.870711381	192.168.56.30	192.168.56.40	TCP	58	56962 → 515 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1758	108.870806955	192.168.56.30	192.168.56.40	TCP	58	56962 → 32774 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1759	108.871524262	192.168.56.30	192.168.56.40	TCP	58	56962 → 2251 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1760	108.871626247	192.168.56.30	192.168.56.40	TCP	58	56962 → 40193 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1761	108.872388035	192.168.56.40	192.168.56.30	TCP	60	515 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1762	108.872388327	192.168.56.40	192.168.56.30	TCP	60	32774 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1763	108.872557074	192.168.56.30	192.168.56.40	TCP	58	56962 → 7938 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1764	108.872660998	192.168.56.30	192.168.56.40	TCP	58	56962 → 668 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1765	108.873653521	192.168.56.30	192.168.56.40	TCP	58	56962 → 65389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1766	108.873763158	192.168.56.30	192.168.56.40	TCP	58	56962 → 1042 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1767	108.875200471	192.168.56.40	192.168.56.30	TCP	60	2251 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1768	108.875200797	192.168.56.40	192.168.56.30	TCP	60	40193 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1769	108.875200907	192.168.56.40	192.168.56.30	TCP	60	7938 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1770	108.875200990	192.168.56.40	192.168.56.30	TCP	60	668 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1771	108.876576641	192.168.56.40	192.168.56.30	TCP	60	65389 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1772	108.876576902	192.168.56.40	192.168.56.30	TCP	60	1042 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1773	108.886929227	192.168.56.30	192.168.56.40	TCP	58	56962 → 5960 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1774	108.887036386	192.168.56.30	192.168.56.40	TCP	58	56962 → 50300 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1775	108.887810749	192.168.56.30	192.168.56.40	TCP	58	56962 → 2525 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1776	108.887940175	192.168.56.30	192.168.56.40	TCP	58	56962 → 22939 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1777	108.888830999	192.168.56.40	192.168.56.30	TCP	60	5960 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1778	108.888831286	192.168.56.40	192.168.56.30	TCP	60	50300 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1779	108.888991903	192.168.56.30	192.168.56.40	TCP	58	56962 → 32775 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1780	108.889091498	192.168.56.30	192.168.56.40	TCP	58	56962 → 5959 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1781	108.889959971	192.168.56.40	192.168.56.30	TCP	60	2525 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

```
▸ Frame 1: Packet, 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on inter  0000  ff ff ff ff ff ff 0a 00  27 00 00 13 08 00 45 00  ····· ····
```

Interprétation : On observe un volume important de paquets TCP générés en très peu de temps, caractéristique d'un scan de ports. Les paquets sont envoyés séquentiellement vers chaque port de la cible.

3.3 Capture Wireshark — Filtre SYN

tcp.flags.syn==1

No.	Time	Source	Destination	Protocol	Length	Info
7	107.659015342	192.168.56.30	192.168.56.40	TCP	58	56962 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
8	107.659142770	192.168.56.30	192.168.56.40	TCP	58	56962 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
9	107.659906439	192.168.56.30	192.168.56.40	TCP	58	56962 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	107.660006489	192.168.56.30	192.168.56.40	TCP	58	56962 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	107.661161793	192.168.56.30	192.168.56.40	TCP	58	56962 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MS
18	107.663976014	192.168.56.30	192.168.56.40	TCP	58	56962 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	107.664079145	192.168.56.30	192.168.56.40	TCP	58	56962 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	107.665245831	192.168.56.30	192.168.56.40	TCP	58	56962 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	107.665344784	192.168.56.30	192.168.56.40	TCP	58	56962 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	107.667227475	192.168.56.30	192.168.56.40	TCP	58	56962 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MS
29	107.671647773	192.168.56.30	192.168.56.40	TCP	58	56962 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	107.671774634	192.168.56.30	192.168.56.40	TCP	58	56962 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	107.672542508	192.168.56.30	192.168.56.40	TCP	58	56962 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	107.672648937	192.168.56.30	192.168.56.40	TCP	58	56962 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	107.678539774	192.168.56.30	192.168.56.40	TCP	58	56962 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	107.678666004	192.168.56.30	192.168.56.40	TCP	58	56962 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	107.679418947	192.168.56.30	192.168.56.40	TCP	58	56962 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	107.680537772	192.168.56.30	192.168.56.40	TCP	58	56962 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
42	107.680639513	192.168.56.30	192.168.56.40	TCP	58	56962 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	107.690549104	192.168.56.30	192.168.56.40	TCP	58	56962 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
48	107.690652897	192.168.56.30	192.168.56.40	TCP	58	56962 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	107.691330329	192.168.56.30	192.168.56.40	TCP	58	56962 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
50	107.691434348	192.168.56.30	192.168.56.40	TCP	58	56962 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
53	107.693083876	192.168.56.30	192.168.56.40	TCP	58	56962 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
54	107.693193900	192.168.56.30	192.168.56.40	TCP	58	56962 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Interprétation : Après application du filtre SYN, seuls les paquets initiateurs de connexion TCP sont visibles. On constate que Kali envoie des SYN vers chaque port de la cible de manière séquentielle ou parallèle selon la configuration du scan.

3.4 Capture Wireshark SYN-ACK

tcp.flags.syn==1 && tcp.flags.ack==1

No.	Time	Source	Destination	Protocol	Length	Info
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460

Interprétation : le syn-ack n'est présent que sur deux ports 80 et 22 ce qui correspond au deux ports ouvert donne par nmap ; donc le syn-ack correspond donc à un port ouvert.

3.5 Capture Wireshark RST

tcp.flags.reset

No.	Time	Source	Destination	Protocol	Length	Info
13	107.662021070	192.168.56.40	192.168.56.30	TCP	60	110 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=6424
15	107.662021299	192.168.56.40	192.168.56.30	TCP	60	113 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
16	107.662050347	192.168.56.30	192.168.56.40	TCP	54	56962 → 80 [RST] Seq=1 Win=0 Len=0
17	107.663427385	192.168.56.40	192.168.56.30	TCP	60	199 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
18	107.663976014	192.168.56.30	192.168.56.40	TCP	58	56962 → 1723 [SYN] Seq=0 Win=1024 Len=0 MS
19	107.664079145	192.168.56.30	192.168.56.40	TCP	58	56962 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS
20	107.665050416	192.168.56.40	192.168.56.30	TCP	60	1723 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0
21	107.665050906	192.168.56.40	192.168.56.30	TCP	60	554 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
22	107.665245831	192.168.56.30	192.168.56.40	TCP	58	56962 → 3306 [SYN] Seq=0 Win=1024 Len=0 MS
23	107.665344784	192.168.56.30	192.168.56.40	TCP	58	56962 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=
24	107.667010822	192.168.56.40	192.168.56.30	TCP	60	3306 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0
25	107.667011086	192.168.56.40	192.168.56.30	TCP	60	21 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Le
26	107.667227475	192.168.56.30	192.168.56.40	TCP	58	56962 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=6424
28	107.669813465	192.168.56.30	192.168.56.40	TCP	54	56962 → 22 [RST] Seq=1 Win=0 Len=0
29	107.671647773	192.168.56.30	192.168.56.40	TCP	58	56962 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS
30	107.671774634	192.168.56.30	192.168.56.40	TCP	58	56962 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS
31	107.672542508	192.168.56.30	192.168.56.40	TCP	58	56962 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS
32	107.672648937	192.168.56.30	192.168.56.40	TCP	58	56962 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=
33	107.674525806	192.168.56.40	192.168.56.30	TCP	60	443 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
34	107.674526171	192.168.56.40	192.168.56.30	TCP	60	445 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
35	107.676703958	192.168.56.40	192.168.56.30	TCP	60	587 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
36	107.676704300	192.168.56.40	192.168.56.30	TCP	60	25 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Le

Interprétation : Pour un port fermé, la réponse est un RST immédiat

4. Analyse technique

- **Un paquet SYN** (Synchronize) est le premier paquet envoyé lors de l'établissement d'une connexion TCP . Il contient le bit SYN activé dans l'en-tête TCP et un numéro de séquence initial (ISN) généré aléatoirement par le client. Dans le contexte d'un scan Nmap (-sS), le paquet SYN est utilisé pour sonder l'état d'un port sans compléter la connexion : si la cible répond par SYN-ACK, le port est ouvert ; si elle répond par RST, le port est fermé ; si elle ne répond pas, le port est filtré.

- **Un port est considéré ouvert** lorsque la cible répond à un SYN par un SYN-ACK, indiquant qu'un service est en écoute et prêt à accepter des connexions. Dans Wireshark, cela se traduit par la séquence : SYN (source → cible) suivi d'un SYN-ACK (cible → source). Nmap interprète ce comportement et clasifie le port comme open dans ses résultats.

- Un scan réseau se reconnaît à plusieurs indicateurs observables dans Wireshark ou un IDS :

  - ✓ Envoi massif et rapide de paquets SYN vers de nombreux ports différents depuis une même IP source

  - ✓ Absence de complétion du handshake TCP (pas de ACK après le SYN-ACK) — caractéristique du scan SYN

  - ✓ Présence d'un grand nombre de RST en réponse, indiquant l'exploration de ports fermés

  - ✓ Intervalle de temps très court entre les paquets (burst de trafic anormal)

5. Utilisation avancée de Wireshark

Filtre 1 : ip.addr == 192.168.56.10

Utilité : Isoler tout le trafic entrant et sortant d'une adresse IP spécifique. Très utile pour se concentrer sur une cible précise parmi un réseau actif.
Exemple concret : Lors du scan de 192.168.56.10, ce filtre permet d'afficher uniquement les échanges entre Kali et Windows, en éliminant le bruit d'autres communications.

Filtre 2 : tcp.port == 80

Utilité : Afficher uniquement le trafic HTTP circulant sur le port 80. Permet d'analyser les requêtes web, détecter des injections ou des communications non chiffrées.

Exemple concret : En capturant le trafic vers un serveur Apache sur la cible Ubuntu, ce filtre révèle les requêtes GET/POST en clair et leur contenu

Filtre 3 : tcp.flags.reset == 1

Utilité : Identifier tous les paquets RST (Reset) échangés sur le réseau. Un nombre élevé de RST est un indicateur fort d'un scan de ports ou d'une tentative de connexion à des services inexistants.

Exemple concret : Après un scan Nmap -sS, ce filtre révèle tous les ports qui ont répondu par RST, confirmant qu'ils sont fermés mais joignables.

6. Lien avec la cybersécurité en entreprise

6.1 Comment un SOC utilise Wireshark

Dans un Security Operations Center (SOC), Wireshark est utilisé principalement pour l'analyse forensique et la validation d'alertes. Lorsqu'un système de détection d'intrusion (IDS/SIEM) génère une alerte, l'analyste SOC utilise Wireshark pour examiner les paquets bruts ayant déclenché cette alerte et confirmer ou infirmer la menace. Il permet également d'analyser des malwares en observant leur comportement réseau (serveurs C2 contactés, protocoles utilisés, exfiltration de données).

6.2 Comment détecter une attaque réseau

La détection d'une attaque réseau dans un environnement d'entreprise repose sur plusieurs indicateurs à surveiller via Wireshark ou un IDS :

- Scan de ports : burst de SYN vers de nombreux ports depuis une même source
- Exfiltration de données : volumes de trafic sortant inhabituels vers des IPs externes inconnues.

7. Conclusion

Ce TP m'a permis de comprendre concrètement le mécanisme du scan TCP SYN et son interprétation au niveau des paquets réseau. J'ai appris à utiliser Nmap pour identifier les services exposés sur une cible et

Wireshark pour observer le trafic généré à chaque étape. La corrélation entre les résultats Nmap et les captures Wireshark m'a donné une vision complète de ce qui se passe réellement lors d'une reconnaissance réseau.

- **Limites de Wireshark :** Il n'effectue pas de détection automatique des menaces ; l'interprétation reste à la charge de l'analyste

À l'issue de ce TP, un scan réseau n'est plus perçu comme une simple commande, mais comme un ensemble d'échanges de paquets structurés dont chaque réponse traduit l'état d'un service et la posture de sécurité d'un hôte. Cette compréhension est fondamentale pour penser comme un analyste SOC ou un pentester professionnel.